

Universidad De San Carlos de Guatemala
Facultad de Ingeniería
Escuela de Ciencias y Sistemas
Lenguajes Formales y de Programación
Sección “B-“



MANUAL TÉCNICO

PROYECTO 2

GUATEMALA
28/10/2021

Angel Miguel García Urizar
201901421

Contenido

Objetivo	2
Ambiente de desarrollo	2
Lenguaje de programación y IDE:	2
Herramientas utilizadas:	2
Acerca de Tkinter:	2
Acerca del programa	3
Acerca del archivo lfp	4
Métodos principales del programa	6
Tabla de Tokens	12
Método de Árbol	12
Expresión Regular	12
Árbol binario	13
Gramática Tipo 2: Gramática Libre del Contexto	16

Objetivo

Que el lector tenga una guía acerca de la información sobre el funcionamiento y el código del programa, para que este le sirva para facilitar correcciones, modificaciones, actualizaciones e implementaciones al programa.

Proporcionar al lector información formal técnica sobre de los procesos llevados a cabo por el programa, explicando para ello las distintas funciones y librerías utilizados y la manera en que éstos se relacionan. Además, brindando información acerca de las herramientas y el entorno en que el programa fue desarrollado.

Ambiente de desarrollo

El programa se desarrolló en Windows 10, con las siguientes herramientas:

Lenguaje de programación y IDE:

- Python 3.8.1
- Visual Studio Code 1.61.2

Herramientas utilizadas:

- HTML5
- CSS
- Tkinter(Librería de Python)

Acerca de Tkinter:

Es una librería ofrecida por Python que permite la construcción de Interfaz Gráfica de Usuario(GUI). La documentación oficial acerca de esta librería se encuentra en el siguiente enlace <https://docs.python.org/3/library/tkinter.html#tkinter-life-preserver>

Acerca del programa

Se requiere un software que permita analizar y realizar reportes de datos para la toma de decisiones futuras y que además se pueda aplicar a cualquier tipo de negocio, generando información interesante para pequeñas empresas. Esta solución debe tener como entrada un lenguaje con cierta estructura por medio de un archivo con extensión “.lfp”

La aplicación cuenta con una interfaz gráfica que posee las siguientes características:

- Cargar archivo: Un botón que al presionarlo permita cargar el archivo con extensión lfp.
- Área de texto: Debe tener un área donde se pueda visualizar y modificar el código lfp.
- Analizar archivo: Un botón que analice el código lfp.
- Consola: Un área de texto que no se pueda editar, solamente visualizar texto generado por las instrucciones dadas por el lenguaje.
- Menú Reportes: Un menú que pueda generar los siguientes reportes:
 - Reporte de Tokens
 - Reporte de Errores
 - Árbol de derivación

Acerca del archivo lfp

```
# Este es un comentario simple(comentario de una línea)
Claves = ["CODIGO", "Producto", "precio_venta", "stock"]

...

Este es un comentario multilínea, se utilizan comillas simples
para hacer un comentario de este tipo
...

Registros = [
    {1, "Camisa", 75.00, 10}
    {2, "Gorra", 50.50, 25}
    {3, "Pantalon", 100, 0}
    {2, "Gorra", 50.50, 25}
    {5, "Zapatos", 150, 500}
]

imprimir("El total de registros es: ");
conteo();
imprimir("El promedio de stock es: ");
promedio();
imprimir("El precio máximo de venta es el siguiente: ");
max("precio_venta");
imprimir("El precio mínimo de venta es el siguiente: ");
min("precio_venta");
imprimir("La cantidad total de producto es: ");
sumar("stock");
imprimir("Existe la siguiente cantidad de productos con el código 2: ");
contarsi("CODIOG", 2);
imprimirln("");
datos();
imprimirln("");
exportarReporte("Reporte de Tienda");
```

1. **Claves:** Inicia con la palabra reservada Claves, seguida del signo igual, luego un corchete de apertura, después un listado de claves y finaliza con un corchete de cierre.
 - a. El listado de claves se compone de cadenas de texto encerradas entre comillas y separadas por coma.
2. **Registros:** Inicia con la palabra reservada Registros, seguida del igual, luego un corchete de apertura, después un listado de Registros y finaliza con un corchete de cierre.
 - a. El listado de registros inicia con una llave de apertura seguida de un conjunto de cadenas de texto encerradas entre comillas y separadas por comas, luego una llave de cierre.

3. Comentarios:

- a. El comentario simple, comentario de una línea inicia con el símbolo '#’.
- b. El comentario multilínea se inicia con tres comillas simples y finaliza con tres comillas simples.

- 4. **imprimir**: La función imprimir inicia con la palabra reservada imprimir seguida de un paréntesis de apertura, luego una cadena de texto seguida de un paréntesis de cierre y finaliza con un punto y coma(;).
- 5. **imprimirln**: Su estructura es la misma que la de la función imprimir, únicamente cambia que inicia con la palabra reservada imprimirln y además su funcionamiento varía un poco, ya que esta da un salto de línea al ejecutarse.
- 6. **conteo**: Se inicia con la palabra reservada conteo seguida de un paréntesis de apertura, continua con un paréntesis de cierre y finaliza con un punto y coma.
- 7. **promedio**: Inicia con la palabra reservada promedio seguida de un paréntesis de apertura, continua con una cadena de texto, luego sigue un paréntesis de cierre y finaliza con un punto y coma.
- 8. **max**: Inicia con la palabra reservada max seguida de un paréntesis de apertura, continua con una cadena de texto, luego sigue un paréntesis de cierre y finaliza con un punto y coma.
- 9. **min**: Inicia con la palabra reservada min seguida de un paréntesis de apertura, continua con una cadena de texto, luego sigue un paréntesis de cierre y finaliza con un punto y coma.
- 10. **sumar**: Inicia con la palabra reservada sumar seguida de un paréntesis de apertura, continua con una cadena de texto, luego sigue un paréntesis de cierre y finaliza con un punto y coma.
- 11. **contarsi**: Inicia con la palabra reservada contarsi seguida de un paréntesis de apertura, continua con una cadena de texto seguida de una coma, luego un valor numérico o una cadena de texto que sigue un paréntesis de cierre y finaliza con un punto y coma.
- 12. **dato**: Se inicia con la palabra reservada conteo seguida de un paréntesis de apertura, continua con un paréntesis de cierre y finaliza con un punto y coma.
- 13. **exportarReporte**: Inicia con la palabra reservada exportarReporte seguida de un paréntesis de apertura, continua con una cadena de texto, luego sigue un paréntesis de cierre y finaliza con un punto y coma.

Métodos principales del programa

Nombre	Descripción	Parámetros
app()	Función principal en la que se crea y ejecuta la interfaz gráfica del programa.	• Ninguno
abrirfile()	Abre un <code>filedialog.askopenfilename</code> y desde aquí el usuario carga el archivo <code>lfp</code> .	• Ninguno
getCodigo()	Obtiene el código ingresado en el área del editor y lo manda al análisis.	• Ninguno
mostrarEnConsola()	Añade al área de consola del programa el resultado de las funciones propias del programa.	• Ninguno
getReporteTokens()	Llama a la función <code>generarReporteTokens</code> y luego abre el reporte generado por dicha función.	• Ninguno
getReporteErrores()	Llama a la función <code>generarReporteErrores</code> y luego abre el reporte generado por dicha función.	• Ninguno
limpiaEditor()	Limpia el área de edición del programa.	• Ninguno
limpiaConsola()	Limpia el área de consola del programa.	• Ninguno
salir()	Cierra el programa.	• Ninguno
getListados()	Función que retorna la lista de claves y la lista de registros.	• Ninguno
getReporteria()	Función devuelve el listado de funciones(comandos propios del programa), que se deben ejecutar.	• Ninguno
generarReporteTokens()	Llama a la función <code>reporteT</code> que se utiliza para generar el reporte de Tokens.	• Ninguno
generarReporteErrores()	Llama a la función <code>reporteE</code> que se utiliza para generar el reporte de Errores.	• Ninguno
isLetra(c)	Función que verifica si el carácter que recibe pertenece a una letra. Para ello utiliza la tabla de valores ASCII.	• <code>c</code> : Carácter a evaluar si corresponde a una letra.
isNumero(c)	Función que verifica si el carácter que recibe pertenece a	• <code>c</code> : Carácter a evaluar si

	un número. Para ello utiliza la tabla de valores ASCII.	corresponde a un número.
isSimbolo(c)	Función que verifica si el carácter que recibe pertenece a un conjunto de símbolos definidos por el lenguaje del programa. Para ello utiliza la tabla de valores ASCII.	• c: Carácter a evaluar si corresponde a un número.
isReservada(palabra)	Función que compara la palabra con una lista de las palabras reservadas definidas por el programa, en busca de coincidencias.	• palabra: Cadena de texto a comparar.
darFormato(entrada, tipo)	Función que se encarga de quitar comillas a las cadenas de texto, convertir a formato int o float, según corresponda.	• entrada: cadena o número a convertir. • tipo: Al tipo que se convertirá la entrada, 'Cadena', 'Número Entero' o 'Número Decimal'.
automata(doc)	Función que lleva a cargo el análisis léxico del contenido del archivo lfp.	• doc: contenido del archivo a analizar.
tokenizar()	Función que se encarga de identificar y almacenar en una lista los distintos tokens que se encontraron durante el análisis.	• Ninguno
parser()	Función que se encarga de llamar a las distintas funciones principales para el análisis sintáctico.	• Ninguno
validarCLAVES()	Verifica que el token actual sea la palabra reservada Claves.	• Ninguno
validarSimboloIgual(parametro, parametro2)	Verifica que el token actual sea el símbolo igual.	• parametro: Token a evaluar. • parametro2: Para indicar a que función debe llamar luego de la verificación.
validarCorchetes(parametro, modo)	Verifica que el token actual sea el correspondiente a un símbolo corchete.	• parametro: Token a evaluar. • parametro2: El carácter con el que se debe hacer

		la comparación: 'a' = [, 'c' =]
validarLClaves()	Verifica que el token actual sea un token cadena.	• Ninguno
validarLClavesP()	Verifica si vienen más claves en el listado de claves.	• Ninguno
validarTkCadena()	Verifica si el token actual es un token cadena.	• Ninguno
validarREGISTROS()	Verifica que el token actual sea la palabra reservada Registros.	• Ninguno
validarLRegistros()	Llama a la función validarRegistro.	• Ninguno
validarRegistro()	Llama a la función validarSimboloLlave().	• Ninguno
validarSimboloLlave(modo)	Función que verifica si el token que recibe por parámetro es el símbolo llave de apertura o de cierre.	• modo: 'a': Para verificar si la llave es de apertura. 'c': Para verificar si la llave es de cierre.
validarLERegistro()	Función que verifica si el token actual es un elemento registro valido y luego que venga una llave de cierre.	• Ninguno
validarElementoRegistro()	Función que valida si el token actual es un token cadena, un token Número Entero o un token Número Decimal.	• Ninguno
validarLERegistroP()	Verifica si vienen más registro en la lista de registros.	• Ninguno
validarParentesis(simbolo, modo)	Valida si el token actual es un símbolo paréntesis.	• simbolo: token actual a analizar. • modo: 'a' = ('c' =)
popPila()	Elimina tokens de la lista hasta encontrar un token palabra reservada.	• Ninguno
validarREPORTES()	Función principal para analizar las funciones(comandos) propios del programa. Llama a la funcion valiarReporte.	• Ninguno

validarReporte()	Valida que el token actual corresponda a una palabra reservada que corresponda a un comando del programa. Según la palabra reservada que sea reconocida, llama a una función específica.	• Ninguno
validarIMPRESION()	Función que se encargar de realizar el análisis sintáctico completo que corresponde al comando 'imprimir' del programa.	• Ninguno
validarIMPRESIONLN()	Función que se encargar de realizar el análisis sintáctico completo que corresponde al comando 'imprimirln' del programa.	• Ninguno
validarCONTEO()	Función que se encargar de realizar el análisis sintáctico completo que corresponde al comando 'conteo' del programa.	• Ninguno
validarPROMEDIO()	Función que se encargar de realizar el análisis sintáctico completo que corresponde al comando 'promedio' del programa.	• Ninguno
validarCONTARSI()	Función que se encargar de realizar el análisis sintáctico completo que corresponde al comando 'contarsi' del programa.	• Ninguno
validarDATOS()	Función que se encargar de realizar el análisis sintáctico completo que corresponde al comando 'datos' del programa.	• Ninguno
validarSUMAR()	Función que se encargar de realizar el análisis sintáctico completo que corresponde al comando 'sumar' del programa.	• Ninguno
validarMAX()	Función que se encargar de realizar el análisis sintáctico completo que corresponde al comando 'max' del programa.	• Ninguno
validarMIN()	Función que se encargar de realizar el análisis sintáctico completo que corresponde al comando 'min' del programa.	• Ninguno
validarEXPORTARREPORTE()	Función que se encargar de realizar el análisis sintáctico completo que corresponde al	• Ninguno

	comando 'exportarReporte' del programa.	
buscarClave(campo)	Función que compara el campo recibido por parámetro con la lista de claves en busca de una coincidencia.	• Ninguno
calcularPromedio(posicion)	Función que calcula el promedio de la posición(número de columna)de la lista de registros.	• posicion: Posición en la lista donde se encontró la clave de coincidencia.
calcularContarsi(posicion, valor)	Cuenta el número de veces que se encuentra el valor dado por parámetro en la posición(número de columna) recibido por parámetro, en la lista de registros.	• posicion: Posición equivalente en la lista de registros a posición de la lista de claves donde se encontró la clave de coincidencia. • valor: Valor con el cual se quiere hacer la comparación puede ser una cadena o un valor numérico.
calcularSuma(posicion)	Suma todos los valores encontrados en la posición(número de columna) de la lista de registros.	• posicion: Posición equivalente en la lista de registros a posición de la lista de claves donde se encontró la clave de coincidencia.
obtenerMax_Min(posicion, modo)	Encuentra el valor máximo o mínimo en la posición(número de columna) de la lista de registros.	• posicion: Posición equivalente en la lista de registros a posición de la lista de claves donde se encontró

		la clave de coincidencia. • modo: 'max' si se quiere encontrar el valor máximo. 'min' si se quiere encontrar el valor mínimo.
<code>exportarReporte(titulo, listaUno, listaDos)</code>	Llama a la función <code>reporteEx</code> .	• titulo: Título que tendrá la tabla de datos. • listaUno: Listado de claves. • listaDos: Listado de registros.
<code>reporteT(lista)</code>	Función que escribe un archivo html donde se muestra la lista de tokens.	• lista: Listado de tokens reconocidos durante el análisis.
<code>reporteE(lista)</code>	Función que escribe un archivo html donde se muestra la lista de errores.	• lista: Listado de errores reconocidos durante el análisis.
<code>reporteT(titulo, claves, registros)</code>	Función que escribe un archivo html donde se muestran en una tabla los datos leídos del archivo lfp.	• titulo: Título de tabla, indicado en la función (comando) <code>exportarReporte</code>. • claves: listado de claves. • registros: listado de registros.

Tabla de Tokens

Token	Descripción informal	Lexema de ejemplo
Identificador	Letra seguida de cero o más letras, dígitos o guion bajo.	Registros, imprimir, exportarReporte, claves
Número Entero	Uno o más dígitos.	12, 100, 1, 37
Número Decimal	Uno o más dígitos, punto, uno o más dígitos.	1.10,0000.5,999.758
Cadenas	Conjunto de caracteres distinto de comilla doble, encerrado entre comillas dobles.	"clave1", "producto","clave_registro"
Símbolo	'=' , '[' , ',' , ']' , '{' , '}' , '(' , ')' , ';' ,	'=' , '[' , ',' , ']' , '{' ,

***NOTA:** Los identificadores Claves, Registros, imprimir, imprimirln, conteo, promedio, contarsi, datos, sumar, max, min, exportarReporte corresponde a palabras reservadas.

Método de Árbol

Expresión Regular

- L = [a-zA-ZñÑ]
- D = [0-9]
- S = ['=' , '[' , ',' , ']' , '{' , '}' , '(' , ')' , ';']
- ^' = Cualquier carácter imprimible menos comilla simple(').
- ^" = Cualquier carácter imprimible menos comilla doble("").
- ^\n = Cualquier carácter imprimible menos salto de línea.
- P = .

ER

- Identificador $\rightarrow L (L \mid D \mid \text{'_'})^*$
- Número $\rightarrow (\text{'+'} \mid \text{'-' }) ? D + (PD +) ?$
- Cadenas $\rightarrow \text{'('} (\text{'^'}) ^* \text{'('}$
- Símbolo $\rightarrow S$
- Comentario Línea $\rightarrow \# (\text{'^'} \backslash n) ^*$
- Comentario Multilínea $\rightarrow \text{'('} (\text{'^'}) ^* \text{'('}$

$(ER) \$ \rightarrow ((((L (L \mid D \mid \text{'_'}) ^*) \mid ((\text{'+'} \mid \text{'-' }) ? D + (PD +) ?)) \mid ((\text{'('} (\text{'^'}) ^* \text{'('} \mid (S))) \mid (((\# (\text{'^'} \backslash n) ^*) \mid (\text{'('} (\text{'^'}) ^* \text{'('}))))) \$$

-

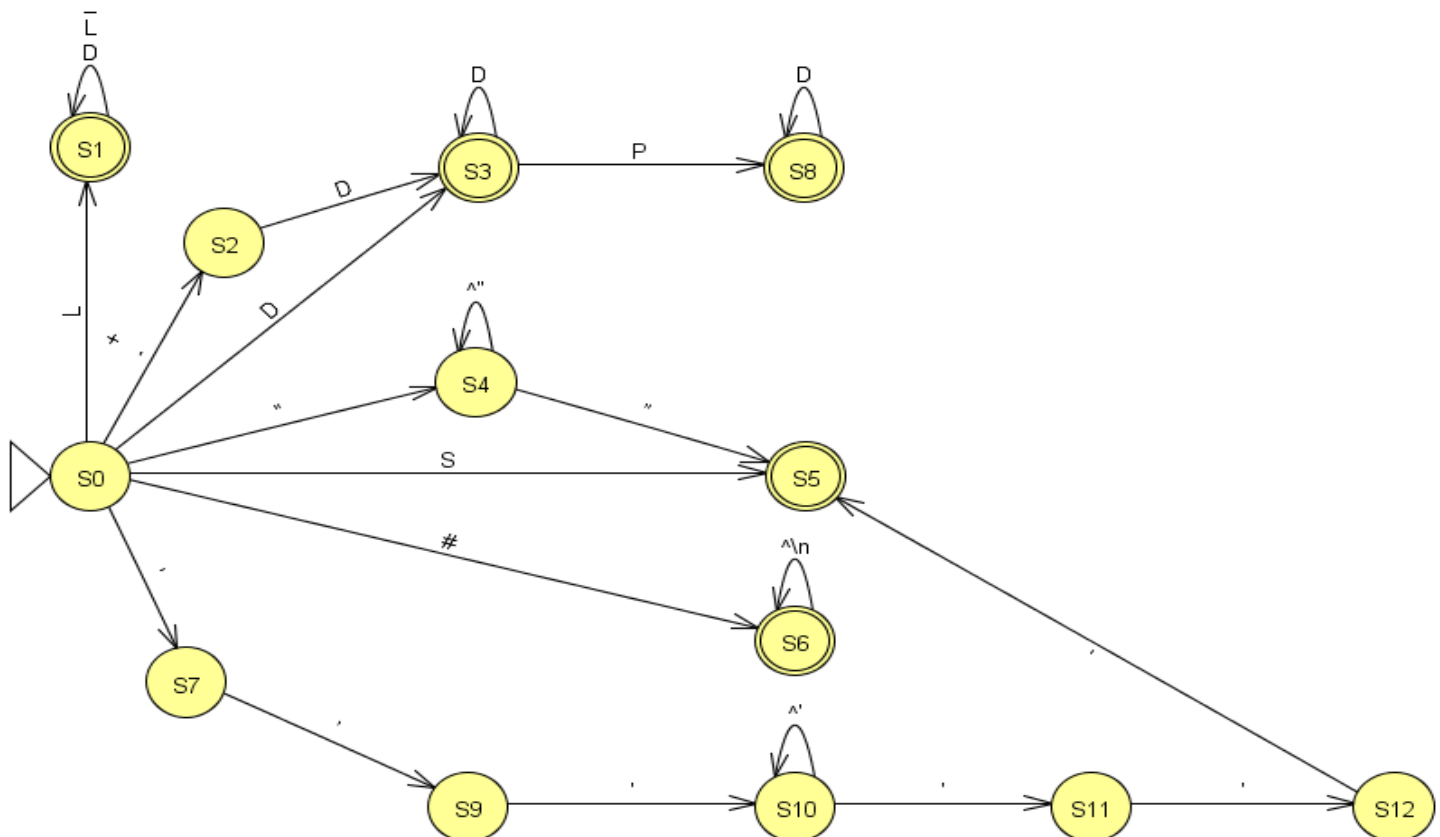
Paso 5: Follow pos

Valor	Hoja	Siguientes
L	1	2,3,4,23
L	2	2,3,4,23
D	3	2,3,4,23
_	4	2,3,4,23
+	5	7
-	6	7
D	7	7,8,23
P	8	9
D	9	9,23
"	10	11,12
^"	11	11,12
"	12	23
S	13	23
#	14	15,23
^\n	15	15,23
'	16	17
'	17	18
'	18	19,20
^'	19	19,20
'	20	21
'	21	22
'	22	23
\$	23	-

Paso 6: Tabla de transiciones.

	Estados	Σ												
		L	D	-	+	-	p	"	^"	\$	#	^\n	'	^'
INICIO	S0	S1	S3	-	S2	S2	-	S4	-	S5	S6	-	S7	-
\$	S1	S1	S1	S1	-	-	-	-	-	-	-	-	-	-
	S2	-	S3	-	-	-	-	-	-	-	-	-	-	-
\$	S3	-	S3	-	-	-	S8	-	-	-	-	-	-	-
	S4	-	-	-	-	-	-	S5	S4	-	-	-	-	-
\$	S5	-	-	-	-	-	-	-	-	-	-	-	-	-
\$	S6	-	-	-	-	-	-	-	-	-	-	S6	-	-
	S7	-	-	-	-	-	-	-	-	-	-	-	S9	-
\$	S8	-	S8	-	-	-	-	-	-	-	-	-	-	-
	S9	-	-	-	-	-	-	-	-	-	-	-	S10	-
	S10	-	-	-	-	-	-	-	-	-	-	-	S11	S10
	S11	-	-	-	-	-	-	-	-	-	-	-	S12	-
	S12	-	-	-	-	-	-	-	-	-	-	-	S5	-

Paso 9: Autómata AFD



Gramática Tipo 2: Gramática Libre del Contexto

Terminales = {TK_Reservada, TK_Simbolo, TK_Cadena, TK_Numero_Entero, TK_Numero_Decimal}

No Terminales = { <INICIO>, <CLAVES>, <REGISTROS>, <INICIO>', <REPORTES>, <LISTACLAVES>, <LISTACLAVES>', <CLAVE>, <LISTAELEMENTOSCLAVE>, <ELEMENTOCLAVE>, <LISTAELEMENTOSCLAVE>', <LISTAREGISTROS>, <REGISTRO>, <LISTAREGISTROS>', <LISTAELEMENTOSREGISTROS>, <ELEMENTOREGISTRO>, <LISTAELEMENTOSREGISTROS>', <REPORTES>', <REPORTE>, <REPORTES>', <IMPRESION>, <IMPRESIONLN>, <CONTEO>, <PROMEDIO>, <CONTARSI>, <DATOS>, <SUMAR>, <MAX>, <MIN>, <EXPORTAR REPORTE> }

Inicio = <INICIO>

Producciones:

<INICIO> ::= <CLAVES> <REGISTROS> <INICIO>'

<INICIO>' ::= <REPORTES>

| EPSILON

<CLAVES> ::= TK_Reservada_Claves TK_Simbolo TK_Simbolo <LISTACLAVES> TK_Simbolo

<LISTACLAVES> ::= <CLAVE> <LISTACLAVES>'

<LISTACLAVES>' ::= <CLAVE> <LISTACLAVES>'

| EPSILON

<CLAVE> ::= <LISTAELEMENTOSCLAVE>

<LISTAELEMENTOSCLAVE> ::= <ELEMENTOCLAVE> <LISTAELEMENTOSCLAVE>'

<LISTAELEMENTOSCLAVE>' ::= TK_Simbolo <ELEMENTOCLAVE>

| EPSILON

<ELEMENTOCLAVE> ::= TK_Cadena

<REGISTROS> ::= TK_Reservada_Registros TK_Simbolo TK_Simbolo <LISTAREGISTROS> TK_Simbolo

<LISTAREGISTROS> ::= <REGISTRO> <LISTAREGISTROS>'

<LISTAREGISTROS>' ::= <REGISTRO> <LISTAREGISTROS>'

| EPSILON

<REGISTRO> ::= TK_Simbolo <LISTAELEMENTOSREGISTROS> TK_Simbolo

<LISTAELEMENTOSREGISTROS> ::= <ELEMENTOREGISTRO> <LISTAELEMENTOSREGISTROS>'

```

<LISTAELEMENTOSREGISTROS>' ::= TK_Simbolo <ELEMENTOREGISTRO>
                                | EPSILON
<ELEMENTOREGISTRO> ::= TK_Cadena
                        | TK_Numero_Entero
                        | TK_Numero_Decimal
<REPORTES> ::= <REPORTE> <REPORTES>'
<REPORTES>' ::= <REPORTE> <REPORTES>'
                | EPSILON
<REPORTE> ::= <IMPRESION>
              | <IMPRESIONLN>
              | <CONTEO>
              | <PROMEDIO>
              | <CONTARSI>
              | <DATOS>
              | <SUMAR>
              | <MAX>
              | <MIN>
              | <EXPORTARREPORTE>
<IMPRESION> ::= TK_Reservada_impresion TK_Simbolo TK_Cadena TK_Simbolo TK_Simbolo
<IMPRESIONLN> ::= TK_Reservada_impresionln TK_Simbolo TK_Cadena TK_Simbolo
TK_Simbolo
<CONTEO> ::= TK_Simbolo TK_Simbolo TK_Simbolo
<PROMEDIO> ::= TK_Reservada_promedio TK_Simbolo TK_Cadena TK_Simbolo TK_Simbolo
<CONTARSI> ::= TK_Reservada_contarsi TK_Simbolo TK_Cadena TK_Simbolo
TK_Numero_Entero TK_Simbolo TK_Simbolo
<DATOS> ::= TK_Reservada_datos TK_Simbolo TK_Simbolo TK_Simbolo
<SUMAR> ::= TK_Reservada_sumar TK_Simbolo TK_Cadena TK_Simbolo TK_Simbolo
<MAX> ::= TK_Reservada_max TK_Simbolo TK_Cadena TK_Simbolo TK_Simbolo
<MIN> ::= TK_Reservada_min TK_Simbolo TK_Cadena TK_Simbolo TK_Simbolo
<EXPORTARREPORTE> ::= TK_Reservada_exportarReporte TK_Simbolo TK_Cadena
TK_Simbolo TK_Simbolo

```